

Document Number: P0802R0

Date: 2017-10-15

Reply-to: Beman Dawes <bdawes at acm dot org>
Nicolai Josuttis <nico at josuttis.de>
Walter E. Brown <webrown.cpp at gmail.com>
Bob Steagall <bob.steagall.cpp at gmail.com>

Audience: Library, Library Evolution

Applying Concepts to the Standard Library [R0]

Abstract

Questions are raised and recommendations made regarding how to best apply [C++ Concepts](#) to the standard library. The most critical recommendation is that concepts for the standard library be based on the [Ranges TS](#) and adopted in two steps: fundamental concepts as soon as possible, followed by other [Ranges TS](#) library changes (a.k.a. STL2) at a more considered pace. Additional recommendation details, background, and rationale are given.

Caution: This is the initial version of this paper; it has not yet been reviewed by LEWG/LWG and its recommendations are subject to change. Updated revisions will be available at wg21.link/p0734.

Background

The Library Extensions Working Group (LEWG) in Toronto asked for a white paper:

- How can [C++ Concepts](#) best be applied to the standard library?
- What policies should the LEWG and LWG define to guide and facilitate such adoption?
- What questions about such adoption warrant further study?
- What other actions are required to facilitate smooth adoption both by the committee and the broad C++ community?

Walter Brown, Nico Josuttis, Bob Steagall, and Beman Dawes volunteered to lead this effort. Eric Niebler and Casey Carter were recruited for their [Ranges TS](#) expertise. *Right now, this paper represents purely the opinions of its authors. It is being circulated for feedback and suggestions from others to be incorporated in later drafts.*

Big Picture Questions

How can the C++ Concepts core language feature be best applied to the standard library?

- It seems clear that the basis for introducing concepts to the standard library must be the [Ranges TS](#). That paper encapsulates the committee's knowledge and experience with fundamental library concepts and how these library concepts can be applied to improve the existing standard library. The [Ranges TS](#) has been implemented and exposed to the C++ community for several years; any other approach would be pure invention and speculation.
- The [Ranges TS](#) has two separable components: a library of fundamental concepts (TS Clauses 6 & 7), and revisions of existing library components (TS Clauses 8-12, also known as STL2). **The characteristics of these two components are quite different, so they should be considered and adopted separately.**

Recommendation: Library Introduction

Clause 20 of the WP, *Library introduction*, should be updated with [Ranges TS](#) clause 6 (*Library introduction*) and the related proposals and issues listed below as soon as a proposal can be prepared and processed by LEWG/LWG. We recommend that Jonathan Wakely and/or other LWG experts well-versed in writing library-wide standardese be recruited to lead this effort.

Rationale: *Library introduction* is where the standard mandates how the rest of the library is specified. Without the concept-related additions to *Library introduction*, adoption of concepts in the library would likely be inconsistent and chaotic.

Related proposals:

- [P0411](#) *Separating Library Requirements and Preconditions* by Jonathan Wakely.
- [P0788](#) *Standard Library Specification in a Concepts and Contracts World* by Walter E. Brown.
- [P0830](#) *Using Concepts and requires in the C++ Standard Library* by Nicolai Josuttis.

Specific issues:

- Specify recommended or required conventions, such as "use CamelCase for concepts".
- Define policies for dealing with concepts that are not simply traits, such as "concepts should be semantics-based, while traits are typically purely syntactic".
- Describe any ABI breaks with concepts/requires. *Private email discussions need to be publicly summarized.*

Recommendation: Fundamental Library Concepts

Ranges TS clause 7 (*Concepts library*) should be adopted by the C++20 WP as soon as a proposal can be prepared and processed by LEWG/LWG. We recommend that Casey Carter and Eric Niebler lead this effort and that they be given sufficient authority to include other fundamental material from the *Ranges TS*.

Rationale: The fundamental concepts are mature and well-known, as they are based on standard library requirements that have been developed and refined from C++98 onward. Because concepts are an entirely new core language feature, these fundamental concepts can be defined in the standard library without breaking any existing C++ code (modulo the usual namespace caveats). Furthermore, failure to standardize these fundamental concepts quickly is likely to result in proliferation of similar but subtly different user-supplied concepts, often with the same names. Confusion seems inevitable under such circumstances.

Recommendation: Mitigate STL2 User-code Breakage

LEWG/LWG should develop a library breakage mitigation approach for STL2, as suggested in [P0684](#), before STL2 can be adopted into the standard library. We recommend that Titus Winters and Howard Hinnant lead this effort.

Rationale: The STL2 portion of the *Ranges TS* introduces numerous improvements over the traditional standard template library (STL). STL2 has the potential to break existing code because it does not exactly duplicate the current requirements of the standard library. Trying to duplicate the current library requirements is not desirable; they are too flawed, and trying to duplicate them exactly contributed to the failure of the C++11 concept effort. Furthermore, STL2 has not yet been deployed in large codebases, so the extent of code breakage is not yet known. In short, STL2 needs more real-world usage experience. Meanwhile, the LEWG/LWG must develop more concepts expertise and an appropriate library breakage mitigation policy, as described in [P0684](#), before STL2 can be integrated into the standard library. To do otherwise risks the stability of user codebases.

Recommendation: Concepts Expertise Development

LEWG/LWG members must increase their expertise and experience with the *Ranges TS*, as should the C++ community as a whole. Action plan and Leads needed.

Rationale: Currently the committee's expertise and experience with C++ concepts and the *Ranges TS* is concentrated in a small number of people, some of whom also have major commitments with other working groups. Yet the full application of concepts to the standard library is going to require significant effort from many people, both inside and outside the committee, if it is to happen reasonably quickly. Focusing attention on this need to build expertise and experience is done on the assumption that the first step in solving a need is to recognize that the need exists.

Recommendation: Solicit *Ranges TS* User Feedback

LEWG/LWG must publicize the need for users to test their codebases, large and small, against the [Ranges TS](#) and report their results and impressions to the committee. Action plan and Leads needed.

Rationale: For reasons similar to those listed above, building expertise and gaining real-world experience using the *Ranges TS* from those outside the committee will provide crucial feedback to help shape the development of library policies regarding concepts.

Question: Should TS Clause 8, *General utilities library*, be part of the fundamental concepts proposal or the STL2 proposal?

To Be Decided

Library Policies for adopting C++ Concepts into the standard library

The "Big Picture" section above aims at enabling the C++ Concepts framework for the standard library. This "Library Policies" section aims to establish library policies to guide C++ Concept usage in proposals for new library components, for modifying existing library components, and for resolving library issues.

Policy: Build the foundations for Concepts first

The standard library will adopt no uses of C++ Concepts until the [Library Introduction](#) and [Fundamental Library Concepts](#) recommendations have been voted into the WP.

Rationale: This is similar to the current LWG/LEWG policy of not adopting use of a core language feature until that feature is actually voted into the WP. Similarly, it is unwise to adopt library motions that depend on library front-matter and fundamental library concepts until they are actually voted into the WP. This applies equally to new library components, existing components, and library issue resolutions.

Policy: Specify [fundamental library concepts](#) where applicable in new library components

New library components should specify [fundamental library concepts](#) rather than traditional requirements where applicable. As always, each library proposal is evaluated on its merits.

Rationale: There is usually no concern that new library components can break existing user code. The primary constraint on use of concepts in new library proposals is simply their lack of familiarity and limited compiler support. Both of these will be resolved as time passes.

Policy: New library components may introduce new C++ concepts

New library component proposals may define new concepts where appropriate. See [How to design good concepts and use them well](#) by Bjarne Stroustrup. As always, each library proposal is evaluated on its merits. Proposal authors should be aware that defining many new concepts may initially cause delays in proposal evaluation and acceptance. As experience with concepts develops, the reverse may occur: it may become difficult to get proposals accepted unless they are specified in terms of concepts.

Policy: Apply C++ concepts to existing library components with great caution

Rationale: [Here be dragons](#). At the moment we simply do not know the extent of user code breakage that will be caused by applying C++ concepts as proposed by the *Ranges TS*. We do know that simply adding constraints via *requires-clauses* as replacements for now-traditional `enable_if` SFINAE is detectable, and that it has the potential to affect user code. (See [enable_if vs. requires: A Case Study](#).)

The best way to slay these dragons is to demonstrate that most do not exist, and that's the point of the [User feedback recommendation](#) above. Where real dragons are actually encountered, [P0648 C++ Stability, Velocity, and Deployment Plans](#) is our best hope for mitigation. But caution is called for in the meantime.
